

Apigee X Simulator

Practice Scenarios for Interview Prep

Three build-it-yourself scenarios - Easy, Medium, and Complex - covering the policy types and platform concepts (API keys, quotas, KVM, OAuth2, JWT, conditional routing, custom faults) that come up most often in an Apigee interview. Each scenario gives you the exact steps to create the proxy in this simulator, the exact policy field values to type in, and sample requests/responses so you know what "correct" looks like before you run it yourself.

Before You Start

Login: admin@apigee-sim.local / Apigee123!

General workflow for every scenario below:

1. Log in, then go to API Proxies > Create Proxy. Fill in Proxy Name, Base Path, and an Initial Target URL (you can add more Target Endpoints later).
2. You land on the proxy's Develop tab. Use "Attach new policy to" to pick PreFlow (request), PostFlow (response), or a Conditional Flow, then click Add and pick a policy type from the picker.
3. Fill in the policy's Configuration tab fields (values given per-scenario below) and click Save Configuration. The XML View tab shows the generated Apigee-style XML and stays in sync with whatever you type on either tab.
4. Repeat for every policy in the scenario, in the order listed - order matters, policies run top to bottom within a phase.
5. Go to the Deploy tab and click Deploy next to an environment (dev is fine for all three scenarios).
6. Go to Trace, pick the proxy + environment, build the request (method, path, headers, query params, body), and click Run Trace. Check the Flow Execution Timeline (or switch to the Diagram view) plus the Request/Response panels against the samples below.

Scenario 1 - Rate-Limited Passthrough Proxy

EASY Basic proxy creation + one traffic-management policy + one mediation policy

What it tests: can you create a proxy end to end, attach a policy in the right phase, and read a Trace result. This is the "can you use the tool at all" baseline question.

Build it

1. Create Proxy: Name RateLimitedHello-v1, Base Path /hello, Initial Target URL https://jsonplaceholder.typicode.com/users.
2. Attach to PreFlow (request): add a SpikeArrest policy.
3. Attach to PostFlow (response): add an AssignMessage policy.
4. Deploy the proxy to dev.

SpikeArrest configuration

Field	Value
Rate	5ps
Identifier (optional)	(leave blank)

AssignMessage configuration

Field	Value
Assign To	response
Action	Add
Target	Headers
Key/Value Pairs	X-Powered-By: Apigee-Simulator

Sample request / response - success

Request

GET /hello
(no headers, no query params, no body)

Response

200 OK
X-Powered-By: Apigee-Simulator
Content-Type: application/json

```
[ { "id": 1, "name": "Leanne Graham", ... }, ... ]
```

Sample request / response - rate exceeded

Run Trace against /hello more than 5 times within one second (just click Run Trace repeatedly and quickly).

Response

429 Too Many Requests

```
{
  "fault": {
    "faultstring": "Spike arrest violation",
    "detail": { "errorcode": "policies.ratelimit.SpikeArrestViolation" }
  }
}
```

In the Timeline, the SpikeArrest step shows status "error" and every step after it (including the Target Request) is skipped - the fault short-circuits the rest of the flow, which is the core Apigee execution model worth being able to explain in an interview.

Scenario 2 - API-Key Protected Catalog with a Quota

MEDIUM API Products/Apps/Developers + VerifyAPIKey + Quota

What it tests: do you understand the API-key/product/app relationship (not just "add a policy"), and can you reason about Quota vs. SpikeArrest (long-window business limit vs. short-window traffic-shaping - a very common interview question).

Build it

1. Go to Developers > Create Developer. Give it any email, e.g. dev1@example.com.

2. Go to API Products > Create Product. Name GoldTier, and add the proxy you're about to create (ProductCatalogSecure-v1) to its Proxies list once the proxy exists - you can also come back and edit the product after step 3.
3. Go to Apps > Register App. Pick the developer from step 1, attach the GoldTier product, and save. The app is auto-approved and immediately shows a Consumer Key and Consumer Secret - copy the Consumer Key, you'll use it as the API key.
4. Create Proxy: Name ProductCatalogSecure-v1, Base Path /catalog, Initial Target URL <https://fakestoreapi.com/products>.
5. Attach to PreFlow (request), in this order: VerifyAPIKey, then Quota.
6. Deploy to dev.

VerifyAPIKey configuration

Field	Value
API Key Location	query
Parameter Name	apikey

Quota configuration

Field	Value
Allow Count	5
Interval	1
Time Unit	minute

Sample request / response - success

Request

```
GET /catalog?apikey=<consumer key from step 3>
```

Response

200 OK

Content-Type: application/json

```
[ { "id": 1, "title": "Fjallraven Backpack", "price": 109.95, ... }, ... ]
```

Sample request / response - missing / invalid key

Request (no apikey)

```
GET /catalog
```

Response

401 Unauthorized

```
{
  "fault": {
    "faultstring": "Failed to resolve API Key variable request.apikey",
    "detail": { "errorcode": "steps.oauth.v2.FailedToResolveAPIKey" }
  }
}
```

Request (wrong apikey)

GET /catalog?apikey=not-a-real-key

Response

401 Unauthorized

```
{
  "fault": {
    "faultstring": "Invalid API Key",
    "detail": { "errorcode": "oauth.v2.InvalidApiKey" }
  }
}
```

Sample request / response - quota exceeded

Run Trace with a valid apikey 6 times within a minute.

Response (6th call)

429 Too Many Requests

```
{
  "fault": {
    "faultstring": "Quota Violation",
    "detail": { "errorcode": "policies.ratelimit.QuotaViolation" }
  }
}
```

Talking point: VerifyAPIKey runs first because there's no point counting quota against an unauthenticated caller - order of operations in PreFlow is deliberate, not arbitrary.

Scenario 3 - OAuth2 + JWT + Multi-Target Routing + Custom Fault

COMPLEX Two proxies, OAuth2 client_credentials, GenerateJWT, Route Rules by HTTP verb, RaiseFault

What it tests: everything above, plus token-based security end to end, minting your own JWT for a downstream call, routing the same base path to different backends by HTTP method, and deliberately rejecting a verb with a custom error - the kind of proxy you'd actually be asked to design in a real Apigee role.

Part A - Token-issuing proxy

1. Reuse the Developer + App from Scenario 2 (or create a new one) - you need its Consumer Key and Consumer Secret.
2. Create Proxy: Name OAuthToken-v2, Base Path /oauth2/token, Initial Target URL <https://jsonplaceholder.typicode.com/users> (unused for this proxy - GenerateAccessToken never calls the target).
3. Attach to PreFlow (request): OAuthV2, Operation = GenerateAccessToken.
4. Deploy to dev.

OAuthV2 configuration (token proxy)

Field	Value
Operation	GenerateAccessToken
Grant Type	client_credentials
Required Scopes	read

Request

POST /oauth2/token

Content-Type: application/json

```
{
  "client_id": "<consumer key>",
  "client_secret": "<consumer secret>",
  "grant_type": "client_credentials"
}
```

Response

200 OK

```
{
  "access_token": "<real, server-issued token>",
  "token_type": "Bearer",
  "expires_in": 3600,
  "scope": "read"
}
```

Copy the access_token - it's a real token this simulator tracks server-side, not a fake string, so it will genuinely be accepted or rejected by Part B below.

Part B - Protected proxy with method-based routing

1. Create Proxy: Name OrdersMulti-v1, Base Path /orders2, Initial Target URL <https://jsonplaceholder.typicode.com/posts> (this becomes your first Target Endpoint, e.g. named "default").
2. On the Develop tab, add a second Target Endpoint named write-target pointing at <https://jsonplaceholder.typicode.com/posts> as well (in a real system this would be a different write-optimized backend - the point here is exercising multiple Target Endpoints).
3. Add Route Rules: rule 1 - Verb GET, Target Endpoint default; rule 2 (the fallback, leave Verb/Path blank) - Target Endpoint write-target.
4. Add a Conditional Flow named block-delete with condition Verb = DELETE. Attach a RaiseFault policy to its request side.
5. Attach to PreFlow (request), in this order: OAuthV2 (Operation = VerifyAccessToken), then GenerateJWT.
6. Attach to PostFlow (response): AssignMessage.
7. Deploy to dev.

OAuthV2 configuration (protected proxy)

Field	Value
Operation	VerifyAccessToken
Access Token Location	Header
Header Name	Authorization

GenerateJWT configuration

Field	Value
Algorithm	HS256
Secret/Key Reference	shared-secret
Issuer	https://accounts.example.com
Audience	orders-internal-api
Expires In (seconds)	300
Assign To Variable	jwt.internal

RaiseFault configuration (block-delete flow)

Field	Value
HTTP Status Code	405
Reason Phrase	Method Not Allowed
Error Payload	{"error": "DELETE is not supported on /orders2"}

AssignMessage configuration (PostFlow)

Field	Value
Assign To	response
Action	Add
Target	Headers
Key/Value Pairs	X-Internal-JWT-Minted: true

Sample request / response - GET routes to the default target

Request

GET /orders2

Authorization: Bearer <access_token from Part A>

Response

200 OK

X-Internal-JWT-Minted: true

```
[ { "userId": 1, "id": 1, "title": "...", "body": "..." }, ... ]
```

In the Timeline: OAuthV2 (success) -> GenerateJWT (success, sets jwt.internal) -> Routing ("Routed GET /orders2 to target endpoint default") -> Target Request -> AssignMessage.

Sample request / response - POST routes to the write target (fallback rule)

Request

POST /orders2

Authorization: Bearer <access_token from Part A>

Content-Type: application/json

```
{ "title": "New Order", "body": "details", "userId": 7 }
```

Response

201 Created

X-Internal-JWT-Minted: true

```
{ "title": "New Order", "body": "details", "userId": 7, "id": 101 }
```

Timeline shows "Routed POST /orders2 to target endpoint write-target" - proof the fallback route rule (no verb specified) caught it since there's no rule explicitly matching POST.

Sample request / response - missing/expired token

Request

GET /orders2
(no Authorization header)

Response

401 Unauthorized

```
{
  "fault": {
    "faultstring": "Invalid Access Token",
    "detail": { "errorcode": "keymanagement.service.invalid_access_token" }
  }
}
```

GenerateJWT, Routing, and Target Request all show as skipped in the Timeline - one more concrete example to point to when explaining fault short-circuiting.

Sample request / response - blocked verb

Request

DELETE /orders2/1
Authorization: Bearer <access_token from Part A>

Response

405 Method Not Allowed

```
{ "error": "DELETE is not supported on /orders2" }
```

This shows RaiseFault used deliberately (not from a failed check) to reject an entire verb at the API-design level, and that a Conditional Flow only runs when its condition matches - OAuthV2/GenerateJWT in PreFlow still run first since PreFlow always runs before any conditional flow.

Interview Talking Points by Scenario

- Easy: proxy lifecycle (create -> attach -> deploy -> trace), PreFlow vs. PostFlow, why a fault skips every later step.
- Medium: API products/apps/developers as the real access-control model behind "just add an API key check"; SpikeArrest (short window, protects infrastructure) vs. Quota (longer window, enforces a business/contract limit) - a classic Apigee interview distinction.
- Complex: client_credentials OAuth2 issuance vs. verification as two different policy configurations of the same policy type; minting an internal JWT to hand to a downstream system after verifying an external

token (token translation, a real integration pattern); Route Rules and multiple Target Endpoints for method-based routing; RaiseFault for deliberate API contract enforcement, not just error handling.

- Across all three: this simulator mirrors real Apigee XML/config 1:1 for every policy used here (VerifyAPIKey, SpikeArrest, Quota, OAuthV2, GenerateJWT, AssignMessage, RaiseFault) - if asked to sketch the policy XML by hand in an interview, you can reconstruct it from the Configuration tab values above.

Part 2 - 30 Practice Test Cases

10 Easy + 10 Medium + 10 Complex - Build Them Yourself

Each test case below is intentionally compact: an objective, a short Build list of exactly what to create/attach/configure, and the sample request(s)/response(s) to check yourself against. Follow the same general workflow from "Before You Start" at the top of this document (create proxy, attach policies, fill Configuration tab fields, deploy, Trace) for every one of them. Try to predict the response BEFORE you run Trace - that prediction is the actual interview skill being practiced here, not the clicking.

10 Easy Test Cases

Easy #1 Stamp a Custom Response Header

Confirm you can attach a Mediation policy to PostFlow and read the result in Trace.

Build:

- Create Proxy: Name HeaderStamp-v1, Base Path /stamp, Target <https://jsonplaceholder.typicode.com/todos/1>.
- Attach to PostFlow (response): AssignMessage - Assign To: response, Action: Add, Target: Headers, Key/Value Pairs: X-Env: sandbox.
- Deploy to dev.

Request

```
GET /stamp
```

Response

```
200 OK
```

```
X-Env: sandbox
```

```
Content-Type: application/json
```

```
{ "userId": 1, "id": 1, "title": "delectus aut autem", "completed": false }
```

Easy #2 Strip a Sensitive Header Before the Target Sees It

Prove you understand AssignMessage's Remove action runs in PreFlow, before the target call.

Build:

- Create Proxy: Name StripAuth-v1, Base Path /strip, Target <https://jsonplaceholder.typicode.com/posts/1>.
- Attach to PreFlow (request): AssignMessage - Assign To: request, Action: Remove, Target: Headers, Key/Value Pairs: X-Internal-Token:..
- Deploy to dev.

Request

```
GET /strip
```

```
X-Internal-Token: secret123
```

Response

200 OK
Content-Type: application/json

```
{ "userId": 1, "id": 1, "title": "...", "body": "..."} }
```

The response body won't visibly change - the point is to open the Target Request panel in Trace and confirm X-Internal-Token is gone before it would have reached the backend.

Easy #3 Allow-List a Single Client IP

Understand AccessControl's ALLOW action and how this simulator's fixed Trace client IP affects the outcome.

Build:

- Create Proxy: Name IpGate-v1, Base Path /secure-ip, Target https://jsonplaceholder.typicode.com/users/1.
- Attach to PreFlow (request): AccessControl - Action: ALLOW, IP List: 203.0.113.1.
- Deploy to dev.

Request

GET /secure-ip

Response (IP List includes 203.0.113.1)

200 OK

```
{ "id": 1, "name": "Leanne Graham", ... }
```

Response (edit the policy: IP List = 198.51.100.9)

403 Forbidden

```
{ "fault": { "faultstring": "Access Denied" } }
```

Every Trace run in this simulator uses a fixed client IP of 203.0.113.1 - that's why the second config (which no longer contains it) gets denied. Knowing the tool's own quirks is fair game to mention in an interview.

Easy #4 Deny-List a Known Bad Actor IP

See the DENY side of AccessControl and contrast it with the ALLOW case above.

Build:

- Create Proxy: Name IpBlock-v1, Base Path /block-ip, Target https://jsonplaceholder.typicode.com/comments/1.
- Attach to PreFlow (request): AccessControl - Action: DENY, IP List: 203.0.113.1.
- Deploy to dev.

Request

GET /block-ip

Response

403 Forbidden

```
{ "fault": { "faultstring": "Access Denied" } }
```

Talking point: ALLOW blocks everyone NOT on the list; DENY blocks only the IPs on the list. Same policy, opposite default posture - a quick way for an interviewer to check you're not just pattern-matching "AccessControl = security policy".

Easy #5 Turn On CORS for a Browser Client

Confirm you know which headers the CORS policy actually sets on the response.

Build:

- Create Proxy: Name CorsDemo-v1, Base Path /cors-demo, Target <https://jsonplaceholder.typicode.com/albums/1>.
- Attach to PreFlow (request): CORS - Allow Origins: *, Allow Methods: GET,POST,OPTIONS, Allow Headers: Content-Type,Authorization.
- Deploy to dev.

Request

GET /cors-demo

Origin: <https://myapp.example.com>

Response

200 OK

Access-Control-Allow-Origin: *

Access-Control-Allow-Methods: GET,POST,OPTIONS

Access-Control-Allow-Headers: Content-Type,Authorization

```
{ "userId": 1, "id": 1, "title": "quidem molestiae enim" }
```

Easy #6 Convert a JSON Response to XML

Confirm you know JSONToXML's Source field controls whether it touches the request or the response.

Build:

- Create Proxy: Name XmlOut-v1, Base Path /xml-out, Target <https://jsonplaceholder.typicode.com/users/2>.
- Attach to PostFlow (response): JSONToXML - Source: response.
- Deploy to dev.

Request

GET /xml-out

Response

200 OK

Content-Type: application/xml

```
<root><id>2</id><name>Ervin  
Howell</name><email>Shanna@melissa.tv</email>...</root>
```

Easy #7 Extract a Field Into a Flow Variable

Confirm `ExtractVariables` reads from Source (request/response) using a `JSONPath` and writes a named flow variable.

Build:

- Create Proxy: Name `ExtractDemo-v1`, Base Path `/extract-demo`, Target `https://jsonplaceholder.typicode.com/users/3`.
- Attach to PostFlow (response): `ExtractVariables` - Source: `response`, `JSONPath`: `$.email`, Variable Name: `extracted.email`.
- Deploy to dev.

Request

GET /extract-demo

Response

200 OK (body unchanged)

```
{ "id": 3, "name": "Clementine Bauch", "email": "Nathan@yesenia.net", ... }
```

The response body is untouched - check the Timeline/Variables panel instead: the `ExtractVariables` step shows `"Extracted $.email -> extracted.email"` with the value `Nathan@yesenia.net`.

Easy #8 Write an Audit Log Line

Confirm `MessageLogging`'s message template placeholders and why it usually belongs in PostFlow, not PreFlow.

Build:

- Create Proxy: Name `LogDemo-v1`, Base Path `/log-demo`, Target `https://jsonplaceholder.typicode.com/posts/5`.
- Attach to PostFlow (response): `MessageLogging` - Syslog Host: `logs.internal.example.com`, Syslog Port: `514`, Message Template: `[{organization.name}] {request.uri} -> {response.status.code}`.
- Deploy to dev.

Request

GET /log-demo

Response

200 OK

Timeline shows: Logged to syslog `logs.internal.example.com:514 -> "[apigee-sim-org] /log-demo -> 200"`. If you attach this same policy to PreFlow instead, `{response.status.code}` resolves empty - the response doesn't exist yet at that point in the flow. That gap is worth being able to explain.

Easy #9 Cache a Response So Repeat Calls Skip the Target

Confirm `ResponseCache` is the one caching policy that automatically skips the target call on a hit.

Build:

- Create Proxy: Name CacheDemo-v1, Base Path /cache-demo, Target <https://jsonplaceholder.typicode.com/photos/1>.
- Attach to PreFlow (request): ResponseCache - TTL Seconds: 60.
- Deploy to dev.

Request (1st call)

GET /cache-demo

Response (1st call)

200 OK
(responsecache.cachehit = false; target actually called)

Request (2nd call, within 60s)

GET /cache-demo

Response (2nd call)

200 OK, identical body
(responsecache.cachehit = true; Target Request step shows skipped)

Easy #10 Enforce a Strict Per-Second Burst Limit

Confirm SpikeArrest's short-window behavior and its fault shape.

Build:

- Create Proxy: Name BurstGuard-v1, Base Path /burst, Target <https://jsonplaceholder.typicode.com/comments?postId=1>.
- Attach to PreFlow (request): SpikeArrest - Rate: 2ps.
- Deploy to dev.

Request (1st and 2nd calls, same second)

GET /burst

Response

200 OK

Request (3rd call, same second)

GET /burst

Response

429 Too Many Requests

```
{
  "fault": {
    "faultstring": "Spike arrest violation",
    "detail": { "errorcode": "policies.ratelimit.SpikeArrestViolation" }
  }
}
```

```
}
```

10 Medium Test Cases

Medium #1 KVM-Backed Basic Auth Gate

Combine KeyValueMapOperations (Get) with BasicAuthentication's "Against Variables" mode - a real decoupled-credentials pattern.

Build:

- Go to KVM (Admin section) > Create Map: Name creds-store, Scope: organization. Add entries username = svcuser and password = svcpass.
- Create Proxy: Name KvmBasicAuth-v1, Base Path /kvm-auth, Target <https://jsonplaceholder.typicode.com/users/4>.
- Attach to PreFlow (request), in order: KeyValueMapOperations (Scope: organization, Map Name: creds-store, Operation: Get, Key: username, Assign To: expected.user); KeyValueMapOperations (same map, Key: password, Assign To: expected.pass); BasicAuthentication (Operation: Decode, Validate Mode: Against Variables, Expected Username Var: expected.user, Expected Password Var: expected.pass).
- Deploy to dev.

Request - correct credentials

GET /kvm-auth

Authorization: Basic c3ZjdXNlcjpwZdmNwYXNz (base64 of svcuser:svcpass)

Response

200 OK

```
{ "id": 4, "name": "Patricia Lebsack", ... }
```

Request - wrong credentials

GET /kvm-auth

Authorization: Basic d3Jvbmc6Y3JlZHM= (base64 of wrong:creds)

Response

401 Unauthorized

```
{ "fault": { "faultstring": "Invalid basic authentication credentials" } }
```

Medium #2 Quota Only on Writes, via a Conditional Flow

Prove Quota doesn't have to sit in PreFlow for everyone - attach it only inside a Conditional Flow that matches a verb.

Build:

- Create Proxy: Name WriteQuota-v1, Base Path /write-quota, Target <https://jsonplaceholder.typicode.com/posts>.

- Add a Conditional Flow named writes, condition Verb = POST. Attach Quota (Allow Count: 3, Interval: 1, Time Unit: minute) to its request step.
- Leave PreFlow/PostFlow empty - GET requests never touch a Quota check at all.
- Deploy to dev.

Request

GET /write-quota (repeat as many times as you like)

Response

200 OK every time - no Quota step runs for GET

Request (4th POST within a minute)

POST /write-quota

Content-Type: application/json

```
{ "title": "x", "body": "y", "userId": 1 }
```

Response (4th POST)

429 Too Many Requests

```
{ "fault": { "faultstring": "Quota Violation", "detail": { "errorcode":  
"policies.ratelimit.QuotaViolation" } } }
```

Medium #3 DecodeJWT: Decode-Only, No Verification

Prove DecodeJWT never checks algorithm, issuer, audience, or expiry - it only requires three dot-separated segments.

Build:

- Create Proxy: Name JwtInspect-v1, Base Path /jwt-inspect, Target <https://jsonplaceholder.typicode.com/users/5>.
- Attach to PreFlow (request): DecodeJWT - Output Variable Prefix: mytoken.
- Deploy to dev.

Request - any structurally valid 3-part token (even an expired/garbage-signature one)

GET /jwt-inspect

Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJleHAiOjEw.eyJleHAiOjEw.eyJleHAiOjEw.anything

Response

200 OK

(mytoken.header and mytoken.claims are set in the Variables panel; DecodeJWT reports success even though exp is long in the past)

Request - malformed token

GET /jwt-inspect

Authorization: Bearer not-a-real-token

Response

401 Unauthorized

```
{ "fault": { "faultstring": "JWT is not valid", "detail": { "errorcode":  
"steps.jwt.InvalidToken" } } }
```

Medium #4 VerifyJWT Enforcing Issuer and Audience

Mint a real token from this simulator's own GenerateJWT, then verify VerifyJWT actually checks issuer/audience/expiry claims.

Build:

- Step 1 - mint a token: any scratch proxy with PreFlow GenerateJWT (Issuer: <https://auth.example.com>, Audience: partner-api, Expires In: 300, Assign To: jwt.mine). Run Trace once and copy the jwt.mine value from the Timeline/Variables panel.
- Step 2 - verify it: Create Proxy: Name JwtStrict-v1, Base Path /jwt-strict, Target <https://jsonplaceholder.typicode.com/users/6>. Attach to PreFlow (request): VerifyJWT - Algorithm: HS256, Issuer: <https://auth.example.com>, Audience: partner-api.
- Deploy JwtStrict-v1 to dev.

Request

GET /jwt-strict

Authorization: Bearer <jwt.mine value copied from step 1>

Response

200 OK

```
{ "id": 6, "name": "Mrs. Dennis Schulist", ... }
```

Response - if you edit VerifyJWT's Audience to something else, e.g. other-api

401 Unauthorized

```
{ "fault": { "faultstring": "JWT audience is invalid", "detail": { "errorcode":  
"steps.jwt.InvalidAudience" } } }
```

Medium #5 Route by Path Suffix to Different Backends

Use two Target Endpoints and Route Rules keyed on Base Path Suffix instead of Verb.

Build:

- Create Proxy: Name TierRouter-v1, Base Path /tier, Initial Target URL <https://jsonplaceholder.typicode.com/posts> (this is the default Target Endpoint).
- Add a second Target Endpoint named vip pointing at <https://jsonplaceholder.typicode.com/users>.
- Add Route Rules: rule 1 - Base Path Suffix /vip, Target Endpoint vip; rule 2 (fallback, leave blank) - Target Endpoint default.
- Deploy to dev.

Request

GET /tier/vip

Response

200 OK - routed to the vip target

```
[ { "id": 1, "name": "Leanne Graham", ... }, ... ]
```

Request

GET /tier

Response

200 OK - routed to the default target (fallback rule)

```
[ { "userId": 1, "id": 1, "title": "...", "body": "..." }, ... ]
```

Medium #6 ServiceCallout to Enrich a Flow Variable

Confirm ServiceCallout makes a real HTTP call and lands the result in a variable, but does NOT automatically merge it into the response.

Build:

- Create Proxy: Name EnrichProxy-v1, Base Path /enrich, Target <https://jsonplaceholder.typicode.com/posts/1>.
- Attach to PreFlow (request): ServiceCallout - Target URL: <https://jsonplaceholder.typicode.com/users/1>, Method: GET, Response Variable: enrichment.user.
- Deploy to dev.

Request

GET /enrich

Response

200 OK - still just the target's own post #1 JSON, unchanged

```
{ "userId": 1, "id": 1, "title": "...", "body": "..." }
```

Check the Timeline: ServiceCallout to <https://jsonplaceholder.typicode.com/users/1> returned 200, and the Variables panel shows enrichment.user populated with the full user object. Merging that into the visible response would need a follow-up AssignMessage or JavaScript step - exactly like real Apigee.

Medium #7 LookupCache / PopulateCache vs. ResponseCache

See why LookupCache/PopulateCache are general-purpose (they don't skip the target on a hit) unlike ResponseCache.

Build:

- Create Proxy: Name ManualCache-v1, Base Path /manual-cache, Target <https://jsonplaceholder.typicode.com/todos/2>.

- Attach to PreFlow (request): LookupCache - Cache Resource: mycache, Cache Key: todo2, Assign To: cached.todo.
- Attach to PostFlow (response): PopulateCache - Cache Resource: mycache, Cache Key: todo2, TTL Seconds: 120.
- Deploy to dev.

Request (1st call)

GET /manual-cache

Response (1st call)

200 OK

(cached.todo = null on the way in - cache miss; PopulateCache writes it afterward)

Request (2nd call, within 120s)

GET /manual-cache

Response (2nd call)

200 OK, same body as before

(cached.todo is now populated with the cached todo object - but the target is STILL called; LookupCache doesn't short-circuit anything by itself)

Medium #8 ConcurrentRateLimit Sanity Check

Know the limits of this simulator: ConcurrentRateLimit is always reported as passing here.

Build:

- Create Proxy: Name ConcurrencyDemo-v1, Base Path /concurrency, Target <https://jsonplaceholder.typicode.com/comments/1>.
- Attach to PreFlow (request): ConcurrentRateLimit - Max Concurrency: 10.
- Deploy to dev.

Request

GET /concurrency

Response

200 OK

(Timeline: "Concurrency simulated within limit of 10")

This simulator doesn't track truly simultaneous in-flight requests from a single manual Trace tool, so ConcurrentRateLimit always reports success. Worth knowing before you spend time trying to "break" it - in real Apigee it throttles based on genuinely concurrent connections, which needs a load-testing tool, not a click-by-click Trace UI, to exercise.

Medium #9 Block an Unsupported Verb via a Conditional Flow

Use RaiseFault inside a Verb-matched Conditional Flow to reject an entire HTTP method by design, not by validation failure.

Build:

- Create Proxy: Name NoUpdates-v1, Base Path /no-updates, Target <https://jsonplaceholder.typicode.com/comments>.
- Add a Conditional Flow named block-put, condition Verb = PUT. Attach RaiseFault (Status: 501, Reason: Not Implemented, Error Payload: {"error": "PUT is not implemented on /no-updates"}) to its request step.
- Deploy to dev.

Request

PUT /no-updates/1

Content-Type: application/json

```
{ "body": "edit" }
```

Response

501 Not Implemented

```
{ "error": "PUT is not implemented on /no-updates" }
```

Request

GET /no-updates

Response

200 OK - normal passthrough, Conditional Flow condition doesn't match GET

Medium #10 AssignVariable: Variable Value vs. Variable Ref

Show the difference between assigning a literal (Variable Value) and copying another variable's live value (Variable Ref).

Build:

- Create Proxy: Name VarDemo-v1, Base Path /var-demo, Target <https://jsonplaceholder.typicode.com/users/7>.
- Attach to PreFlow (request), in order: AssignMessage (Action: AssignVariable, Variable Name: source.var, Variable Value: hello-from-step-1); AssignMessage (Action: AssignVariable, Variable Name: copied.var, Variable Ref: source.var).
- Deploy to dev.

Request

GET /var-demo

Response

200 OK

```
{ "id": 7, "name": "Kurtis Weissnat", ... }
```

Variables panel: `source.var = "hello-from-step-1"` (a literal from Variable Value), `copied.var = "hello-from-step-1"` too (copied live from `source.var` via Variable Ref, not typed in again). This is the standard answer to "how do I copy one flow variable into another" in an Apigee interview.

10 Complex Test Cases

Complex #1 Dual-Factor Gate: API Key + KVM-Backed Basic Auth

Stack VerifyAPIKey and a KVM/BasicAuthentication check in one PreFlow, and reason about the short-circuit order.

Build:

- Reuse the Developer + App + GoldTier product from Scenario 2 (or create new ones) for a Consumer Key. Reuse the creds-store KVM from Medium #1 (or recreate it: username=svcuser, password=svcpass, scope organization).
- Create Proxy: Name DualAuth-v1, Base Path /dual-auth, Target `https://jsonplaceholder.typicode.com/users/8`.
- Attach to PreFlow (request), in order: VerifyAPIKey (API Key Location: query, Parameter Name: apikey); KeyValueTypeOperations x2 (Get username/password from creds-store into expected.user/expected.pass); BasicAuthentication (Operation: Decode, Validate Mode: Against Variables, Expected Username Var: expected.user, Expected Password Var: expected.pass).
- Deploy to dev.

Request - valid key + valid Basic creds

```
GET /dual-auth?apikey=<consumer key>
Authorization: Basic c3ZjdXNlcjpwZmNwYXNz
```

Response

200 OK

```
{ "id": 8, "name": "Nicholas Runolfsson", ... }
```

Request - valid key, wrong Basic creds

```
GET /dual-auth?apikey=<consumer key>
Authorization: Basic d3Jvbmc6Y3JlZHM=
```

Response

401 Unauthorized

```
{ "fault": { "faultstring": "Invalid basic authentication credentials" } }
(Timeline: VerifyAPIKey succeeded first, THEN BasicAuthentication failed)
```

Request - missing apikey entirely

```
GET /dual-auth
Authorization: Basic c3ZjdXNlcjpwZmNwYXNz
```

Response

401 Unauthorized

```
{ "fault": { "faultstring": "Failed to resolve API Key variable request.apikey" } }
```

(BasicAuthentication never even runs - it's skipped once VerifyAPIKey faults)

Complex #2 OAuth2-Protected Catalog with Quota and Verb-Scoped Caching

Combine token verification, a business-limit Quota, and a ResponseCache that only applies to GET via a Conditional Flow.

Build:

- Reuse the token-issuing OAuthToken-v2 proxy from Scenario 3 Part A (or rebuild it) to obtain a real access_token.
- Create Proxy: Name SecureCatalog-v1, Base Path /secure-catalog, Target <https://fakestoreapi.com/products>.
- Attach to PreFlow (request), in order: OAuthV2 (Operation: VerifyAccessToken); Quota (Allow Count: 10, Interval: 1, Time Unit: hour).
- Add a Conditional Flow named cache-get, condition Verb = GET. Attach ResponseCache (TTL Seconds: 30) to its request step.
- Deploy to dev.

Request (1st GET, valid token)

GET /secure-catalog
Authorization: Bearer <access_token>

Response (1st GET)

200 OK - cache miss, real target call

```
[ { "id": 1, "title": "Fjallraven Backpack", ... }, ... ]
```

Request (2nd GET within 30s, same token)

GET /secure-catalog
Authorization: Bearer <access_token>

Response (2nd GET)

200 OK - served from cache, target skipped, same body

Request - missing token

GET /secure-catalog

Response

401 Unauthorized

```
{ "fault": { "faultstring": "Invalid Access Token" } }
```

(Quota never runs - unauthenticated calls don't consume the app's quota)

Complex #3 Public Browser API: Full Traffic-Management Stack

Prove CORS headers survive a later PreFlow fault, since faultResponse only overwrites status/reason/body - never headers already set.

Build:

- Create Proxy: Name PublicWidget-v1, Base Path /widget, Target <https://jsonplaceholder.typicode.com/photos>.
- Attach to PreFlow (request), in order: CORS (Allow Origins: *, Allow Methods: GET, Allow Headers: Content-Type); SpikeArrest (Rate: 3ps); Quota (Allow Count: 8, Interval: 1, Time Unit: minute).
- Deploy to dev.

Request

```
GET /widget
Origin: https://myapp.example.com
```

Response

```
200 OK
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET

[ { "albumId": 1, "id": 1, "title": "...", "url": "..." }, ... ]
```

Response (4th call within the same second - SpikeArrest trips)

```
429 Too Many Requests
Access-Control-Allow-Origin: * <- still present!
Access-Control-Allow-Methods: GET <- still present!

{ "fault": { "faultstring": "Spike arrest violation" } }
```

The CORS headers survive because CORS already ran (and wrote directly to ctx.response.headers) before SpikeArrest faulted two steps later - a fault only replaces status/reasonPhrase/body, never headers set by earlier successful steps. Good to know cold in an interview.

Complex #4 Org-Wide Enforcement via a Shared Flow + Flow Hook

Enforce a policy on every proxy in the org without editing each proxy - the real reason Flow Hooks exist.

Build:

- Go to Shared Flows > Create Shared Flow named common-security. Add a single VerifyAPIKey policy to it (API Key Location: query, Parameter Name: apikey).
- Go to Flow Hooks (Admin section) and set PreProxyFlowHook = common-security.
- Create Proxy: Name HookedProxy-v1, Base Path /hooked, Target <https://jsonplaceholder.typicode.com/users/9>. Leave its own PreFlow empty on purpose.
- Deploy to dev.

Request - no apikey

```
GET /hooked
```

Response

401 Unauthorized

```
{ "fault": { "faultstring": "Failed to resolve API Key variable request.apikey" } }
```

(enforced by the Flow Hook - HookedProxy-v1 itself has zero PreFlow policies)

Request - valid apikey

GET /hooked?apikey=<consumer key from any earlier scenario>

Response

200 OK

```
{ "id": 9, "name": "Glenna Reichert", ... }
```

This is exactly how a platform team would roll out mandatory auth, logging, or CORS across dozens of proxies overnight - a very common "how would you..." system-design question.

Complex #5 Simulated Connector, Then a Real Network Failure

Contrast the canned connector policies (always succeed, deterministic fake data) with ServiceCallout, the one policy here that makes a genuine outbound HTTP call and can genuinely fail.

Build:

- Create Proxy: Name CrmSync-v1, Base Path /crm-sync, Target <https://jsonplaceholder.typicode.com/users/10>.
- Attach to PreFlow (request), in order: SalesforceConnector (Operation: Update, SObject: Contact); ServiceCallout (Target URL: <https://this-domain-does-not-exist-12345.example>, Method: GET, Response Variable: callout.result).
- Deploy to dev.

Request

GET /crm-sync

Response

502 Bad Gateway

```
{ "fault": { "faultstring": "ServiceCallout request failed" } }
```

Timeline: SalesforceConnector succeeds ("Salesforce Update on Contact simulated", pure canned data, no real network call) - then ServiceCallout genuinely tries to reach an unreachable host and fails with a real 502, short-circuiting the Target Request and PostFlow entirely. Being able to name which policies in a catalog are "real network calls" vs. "simulated/canned" is a good sign of hands-on familiarity.

Complex #6 Legacy XML Adapter: XML In, JSON Internally, XML Out

Chain XMLToJSON on the request with JSONT/XML on the response to bridge a legacy XML client to a JSON backend.

Build:

- Create Proxy: Name LegacyAdapter-v1, Base Path /legacy, Target <https://jsonplaceholder.typicode.com/posts>.
- Attach to PreFlow (request): XMLToJSON - Source: request.
- Attach to PostFlow (response): JSONToXML - Source: response.
- Deploy to dev.

Request

POST /legacy

Content-Type: application/xml

```
<order><title>Widget</title><body>details</body><userId>3</userId></order>
```

Response

201 Created

Content-Type: application/xml

```
<root><title>Widget</title><body>details</body><userId>3</userId><id>101</id></root>
```

XMLToJSON turns the incoming XML into a plain object (values come through as strings - a simple converter doesn't infer number types) before it's sent to the JSON backend; JSONToXML then wraps whatever the backend returns back into <root>...</root> for the legacy caller. This request/response protocol-translation pattern is a realistic Apigee use case for SOAP/XML clients in front of modern JSON services.

Complex #7 Cache-Aside Pattern: Why LookupCache/PopulateCache Aren't a Drop-In Replacement for ResponseCache

Build a deliberately GET-only cache-aside proxy and confirm the target still gets called on a LookupCache hit - the opposite of ResponseCache.

Build:

- Create Proxy: Name CacheAside-v1, Base Path /cache-aside, Target <https://jsonplaceholder.typicode.com/albums/3>.
- Add a Conditional Flow named reads, condition Verb = GET. Attach LookupCache (Cache Resource: albumCache, Cache Key: album3, Assign To: cached.album) to its request step, and PopulateCache (Cache Resource: albumCache, Cache Key: album3, TTL Seconds: 45) to its response step.
- Add a Conditional Flow named blocked-writes, condition Verb = DELETE. Attach RaiseFault (Status: 405, Reason: Method Not Allowed, Error Payload: {"error": "DELETE not supported - read-through cache only"}) to its request step.
- Deploy to dev.

Request (1st GET)

GET /cache-aside

Response (1st GET)

200 OK - cached.album = null going in (miss); PopulateCache writes it after the target call

Request (2nd GET, within 45s)

GET /cache-aside

Response (2nd GET)

200 OK, same body - cached.album IS populated this time (hit), but the target is called again anyway
(unlike ResponseCache's automatic skip, nothing here short-circuits the target call on your behalf)

Request

DELETE /cache-aside

Response

405 Method Not Allowed

```
{ "error": "DELETE not supported - read-through cache only" }
```

Complex #8 Capstone: Design and Justify the PreFlow Ordering

Combine six policy types in one proxy and be ready to defend WHY each one is ordered where it is - a design question, not just a build-it exercise.

Build:

- Create Proxy: Name SearchAPI-v1, Base Path /search, Target https://jsonplaceholder.typicode.com/users.
- Attach to PreFlow (request), in this exact order: CORS; SpikeArrest (Rate: 5ps); VerifyAPIKey (API Key Location: query, Parameter Name: apikey); Quota (Allow Count: 20, Interval: 1, Time Unit: hour); ResponseCache (TTL Seconds: 15).
- Attach to PostFlow (response): AssignMessage (Assign To: response, Action: Add, Target: Headers, Key/Value Pairs: X-RateLimit-Policy: quota+spike-arrest).
- Deploy to dev.

Request - valid apikey, 1st call

GET /search?apikey=<consumer key from any earlier scenario>

Response

200 OK

X-RateLimit-Policy: quota+spike-arrest

```
[ { "id": 1, "name": "Leanne Graham", ... }, ... ]
```

Request - missing apikey

GET /search

Response

401 Unauthorized - Quota and ResponseCache never run

Be ready to explain: SpikeArrest sits BEFORE VerifyAPIKey because infrastructure protection has to apply even to unauthenticated bursts, while Quota sits AFTER VerifyAPIKey because it's a per-app business limit that shouldn't count against callers who were never authenticated at all. That distinction, stated out loud, is one of the most common Apigee interview questions there is.

Complex #9 PostFlow Still Runs After a Fault - Audit Logging That Never Gets Skipped

Confirm ProxyEndpoint PostFlow (Response) executes even when PreFlow already faulted - real Apigee's DefaultFaultRule behavior.

Build:

- Create Proxy: Name AuditedGate-v1, Base Path /audited, Target <https://jsonplaceholder.typicode.com/comments/1>.
- Attach to PreFlow (request): RaiseFault (Status: 403, Reason: Forbidden, Error Payload: {"error": "temporarily disabled for maintenance"}) - unconditional, always fires.
- Attach to PostFlow (response): MessageLogging (Syslog Host: audit.example.com, Syslog Port: 514, Message Template: [{organization.name}] {request.uri} -> {response.status.code}).
- Deploy to dev.

Request

```
GET /audited
```

Response

```
403 Forbidden
```

```
{ "error": "temporarily disabled for maintenance" }
```

Timeline: PreFlow RaiseFault (error) -> Routing/Target Request/TargetEndpoint PostFlow all show skipped -> but ProxyEndpoint PostFlow's MessageLogging still runs and logs "[apigee-sim-org] /audited -> 403" - the correct final status, because ctx.response.status was already set by the fault before PostFlow ran. Builds directly on the header-survival behavior from Complex #3, from a different angle: not just data survives a fault, entire PostFlow policies still execute.

Complex #10 Open-Ended: Design Your Own Three-Tier Proxy

A true system-design exercise, not a fixed-answer Trace check - the kind of question an interviewer asks when they want to see how you think, not just whether you can follow steps.

Build:

- Pick a fictional use case (e.g. a partner-facing orders API, a mobile app's search endpoint, an internal admin tool).
- Design one proxy that uses at least six different policy types from the full catalog (Traffic Management, Security, Mediation, Extension - mix categories, don't just stack three rate limiters).
- Justify, in your own words, the phase (PreFlow/PostFlow/Conditional Flow/Target Endpoint) and order of every policy you chose.
- Build it for real in this simulator, deploy it, and Trace at least three request variations: one clean success, one that should fail security, and one that should hit a business limit (Quota) or a deliberate design rule (RaiseFault).

Self-check rubric: did you separate traffic-shaping (SpikeArrest) from business limits (Quota)? Did security policies run before anything that costs money or capacity to execute? Did you use a Conditional Flow only where the

condition genuinely differs by verb or path, rather than everywhere out of habit? Can you sketch the policy XML for at least two of your policies from memory, the way the Configuration tab would generate it?

Wrap-Up

If you can build all 30 of these from the Build list alone (without peeking at the sample request/response until after you've predicted it), you've covered the full breadth of this simulator's policy catalog: every Traffic Management policy, every Security policy, every Mediation/transform policy, the connector and Extension policies, KVM, Shared Flows, Flow Hooks, multiple Target Endpoints, Route Rules, and Conditional Flows - which is the same breadth most Apigee interviews probe, just usually one or two policies at a time instead of thirty.